

Temporal Neural Networks: A Dynamical Systems Approach to Stable and Robust Neural Computation

Edward Chalk
Independent Researcher
edward@fleetingswallow.com
<https://github.com/pcoz>

January 2025

(revised: added self-tuning latency feedback, Section 5)

Abstract

We present Temporal Neural Networks (TNNs), a neural architecture in which each neuron is modeled as a continuous-time dynamical system rather than an instantaneous function. Unlike classical feedforward neural networks that compute outputs as $y = f(Wx + b)$, TNN neurons evolve according to first-order differential equations $dV/dt = (1/\tau)(-V + f(Wx + b))$, introducing internal state, temporal inertia, and continuous-time dynamics at the neuron level.

TNNs are constructed using a three-phase pipeline: (1) conventional feedforward training to learn representations, (2) post-hoc dynamical system identification via symbolic regression to characterize temporal behavior, and (3) conversion to continuous-time temporal form. This decoupling of representation learning from temporal dynamics distinguishes TNNs from recurrent neural networks, continuous-time RNNs, Neural ODEs, and state space sequence models.

We evaluate TNNs on the UCI Human Activity Recognition benchmark using subject-independent splits. TNNs match classical accuracy (95.1% vs 95.3%) while exhibiting dramatically improved temporal stability (75–91% fewer prediction flips under noise) and superior robustness to missing data (+8.4% accuracy at 40% feature dropout). Ablation studies demonstrate that these gains arise from implicit dynamical regularization that constrains trajectory volatility rather than weights, and that TNN dynamics provide benefits beyond what post-hoc smoothing or noise-injection training can achieve alone. Finally, we introduce an optional *self-tuning latency feedback* that lets each neuron adapt its own time constant online—recovering both the stability of a large τ and the responsiveness of a small one—using a self-calibrating change detector whose internal trace doubles as a label-free characterization of the signal (change-points and noise floor).

1 Introduction

Classical artificial neural networks model neurons as instantaneous, memoryless functions. Given an input vector x , the network computes $y = f(Wx + b)$ with no notion of time beyond discrete evaluation steps. This abstraction has proven effective for static pattern recognition but diverges sharply from biological neural computation, where neurons integrate inputs over time, exhibit leakage toward resting potential, and possess intrinsic time constants [Gerstner and Kistler, 2002].

Temporal behavior in modern machine learning is typically introduced through architectural recurrence (e.g., RNNs, LSTMs, GRUs) or by treating network depth as a proxy for time (e.g., Neural ODEs). In contrast, Temporal Neural Networks (TNNs) introduce temporal dynamics directly at the neuron level while preserving a feedforward computational graph.

Each TNN neuron evolves according to:

$$\frac{dV}{dt} = \frac{1}{\tau} (-V + f(Wx + b)) \quad (1)$$

where V is the neuron’s membrane potential (state), τ is the time constant governing temporal responsiveness, and the neuron *evolves toward* its target activation rather than jumping instantaneously.

This formulation introduces temporal inertia without recurrence, enabling robustness to noise and missing data while remaining compatible with standard backpropagation and inference pipelines.

1.1 Contributions

1. A three-phase pipeline that separates representation learning from temporal dynamics, enabling temporal conversion of pre-trained networks
2. A system-identification approach using symbolic regression (PPF) to extract interpretable neuron-level dynamics
3. Empirical evidence of large stability and robustness gains without loss of accuracy
4. Comprehensive ablation studies characterizing the effect of time constants, comparing against post-hoc smoothing baselines, and evaluating interaction with noise-injection training
5. A dynamical interpretation of robustness as implicit trajectory-level regularization
6. An optional self-tuning latency feedback (adaptive time constant) that resolves the stability–responsiveness trade-off of a fixed τ via a self-calibrating, closed-loop change detector, and whose trace serves as a free online signal characterization

2 Related Work

Temporal computation in neural networks has extensive prior art spanning computational neuroscience and machine learning. We position TNNs relative to this literature, emphasizing that our contribution is not the invention of temporal neural computation, but rather a specific methodology for introducing neuron-level temporal dynamics as a post-hoc stability mechanism.

2.1 Recurrent Neural Networks

Discrete-time recurrent models (RNN, LSTM, GRU) introduce time through explicit recurrence in hidden state updates [Hochreiter and Schmidhuber, 1997, Cho et al., 2014]. While expressive, these models learn temporal state transitions jointly with representations, often resulting in sensitivity to noise, hidden-state instability, and brittle behavior under partial observability.

TNNs differ fundamentally: recurrence is not used, and temporal dynamics are not learned jointly with representations. Instead, learned representations are held fixed while dynamics are introduced through explicit, low-order temporal models.

2.2 Continuous-Time Recurrent Neural Networks

Continuous-Time RNNs (CTRNNs) model neural state evolution using differential equations and have been studied extensively in computational neuroscience and dynamical systems theory [Beer, 1995]. CTRNNs exhibit rich dynamical behaviors including limit cycles and chaotic attractors, but are typically trained end-to-end as dynamical systems with recurrent coupling, resulting in highly parameterized and difficult-to-interpret models.

TNNs retain feedforward topology and introduce temporal dynamics post-hoc as a stability constraint rather than as a learned recurrent transition function. This makes TNNs closer to adding a “dynamical constraint layer” than to training a full CTRNN.

2.3 Neural ODEs and Latent Continuous-Time Models

Neural Ordinary Differential Equations reinterpret network depth as continuous time, replacing discrete layers with differential equation solvers [Chen et al., 2018]. Extensions including ODE-RNNs, Latent ODEs [Rubanova et al., 2019], GRU-ODE-Bayes [De Brouwer et al., 2019], and Neural Controlled Differential Equations [Kidger et al., 2020] model continuous-time latent dynamics for irregularly sampled data.

These approaches focus on sequence modeling and typically require numerical ODE solvers at inference time. TNNs do not treat depth as time and require no solver-heavy inference—temporal dynamics arise from simple leaky integration at the neuron level, making TNN inference computationally inexpensive and predictable.

2.4 Liquid Time-Constant Networks

Liquid Time-Constant Networks (LTCs) introduce state-dependent time constants within continuous-time recurrent architectures [Hasani et al., 2022]. LTCs are trained end-to-end as continuous-time RNNs and typically require differential equation solvers.

TNNs are complementary: rather than learning dynamics end-to-end, TNNs apply a post-hoc temporalization transform to existing feedforward networks. Time constants can be identified via system identification (Phase 2) or set uniformly, without requiring solver-based inference.

2.5 State Space Sequence Models

Modern state space sequence models such as S4 [Gu et al., 2022] and Mamba [Gu and Dao, 2023] frame long-range dependency modeling via structured dynamical systems with efficient computation. These are sequence modeling backbones designed to learn long-range dependencies.

TNNs are not a sequence model backbone but rather a unit-dynamics stability mechanism that can be applied to a wide class of models, including those not designed as sequence backbones.

2.6 Robustness and Regularization

Robustness in neural networks is commonly addressed through explicit regularization and training-time augmentation. Key approaches include:

- **Noise injection:** Adding noise during training induces regularization effects related to Jacobian penalties [Rifai et al., 2011, Bishop, 1995]
- **Adversarial training:** Framing robustness as minimax optimization [Madry et al., 2018]

- **Lipschitz constraints:** Spectral normalization and related techniques constrain operator norms [Miyato et al., 2018]
- **Dropout:** Randomly zeroing activations during training [Srivastava et al., 2014]

These methods constrain weights, gradients, or training distributions. TNNs differ by constraining *state trajectories* at inference time, acting as implicit temporal regularization. This is complementary to classical regularization—the mechanisms operate on different axes.

3 Method

3.1 Overview: Three-Phase Pipeline

The TNN pipeline separates representation learning from temporal dynamics:

1. **Phase 1: Classical Training**—Train a standard feedforward network using backpropagation
2. **Phase 2: Dynamical System Identification**—Characterize temporal behavior via symbolic regression (optional but recommended for interpretability)
3. **Phase 3: Temporal Conversion**—Convert neurons to temporal form with explicit leaky integration

This separation is intentional: it allows temporal conversion of *any* pre-trained feedforward network without retraining.

3.2 Phase 1: Classical Training

We first train a standard feedforward neural network using backpropagation:

$$h^{(l)} = \sigma(W^{(l)}h^{(l-1)} + b^{(l)}) \quad (2)$$

This phase learns representations without temporal dynamics. Any standard training procedure, optimizer, and regularization technique can be used.

3.3 Phase 2: Dynamical System Identification

Phase 2 is a *system identification* step, not a learning step. The goal is to characterize the temporal behavior of the trained network when processing sequential data.

3.3.1 Data Collection

As the trained network processes temporally ordered input streams, we record neuron activation trajectories $a_i(t) = \sigma(W_i x(t) + b_i)$. Inputs are unmodified; no smoothing or temporal augmentation is applied.

3.3.2 Objective

The goal is to identify the simplest continuous-time dynamical system whose trajectories approximate the observed activations:

$$\dot{V}(t) = g(V(t), a(t)) \quad (3)$$

subject to:

- Stability (bounded solutions)
- Low-order dynamics
- Interpretability

3.3.3 Symbolic Regression via PPF

We apply Partial Form Finder (PPF), a constrained symbolic regression system optimized for temporal signals¹. Unlike unconstrained symbolic regression, PPF restricts the hypothesis space to biologically and physically plausible forms:

- Linear leakage: $\dot{V} = -(V - V_{\text{rest}})/\tau + I$
- Exponential decay: $V(t) = Ae^{-t/\tau} + B$
- Damped oscillations: $V(t) = Ae^{-\zeta\omega t} \sin(\omega t + \phi)$
- Saturating responses

Candidate models are evaluated based on:

- R^2 on held-out trajectories
- Stability under extrapolation
- Parameter consistency across samples

This bias toward simplicity prevents overfitting and favors robust, interpretable dynamics.

3.3.4 Outputs

Phase 2 produces:

- Time constants τ (per-neuron or per-layer)
- Optional damping parameters
- Confidence bounds on parameters

Crucially, **weights are not modified**—Phase 2 only identifies temporal parameters.

¹<https://github.com/pcoz/timeseries-formula-finder>

3.4 Phase 3: Temporal Conversion

Each neuron is converted to temporal form using explicit Euler integration:

$$V \leftarrow V + \frac{dt}{\tau}(\text{target} - V) \quad (4)$$

where $\text{target} = \sigma(Wh + b)$ is the classical activation.

Time constants may be:

- Uniform across the network (simplest)
- Layer-specific
- Per-neuron (from PPF-derived identification)
- Optionally fine-tuned via gradient descent

3.5 Inference

Inference proceeds via a fixed number of “settle steps”:

Algorithm 1 TNN Inference

```

Initialize all neuron states  $V \leftarrow 0$ 
for  $t = 1$  to  $T_{\text{settle}}$  do
  for each layer  $l$  do
     $\text{target} \leftarrow \sigma(W^{(l)}h^{(l-1)} + b^{(l)})$ 
     $V^{(l)} \leftarrow V^{(l)} + \frac{dt}{\tau}(\text{target} - V^{(l)})$ 
  end for
end for
return  $\arg \max(V^{(L)})$ 

```

For streaming applications, the network maintains state across inputs rather than resetting.

3.6 Temporal Dynamics as Implicit Regularization

TNNs impose an implicit regularization on trajectory volatility rather than weights. The dynamics penalize rapid state changes:

$$\int \|\dot{V}(t)\|^2 dt \quad (5)$$

This acts as a continuous-time smoothing constraint that suppresses high-frequency perturbations without reducing representational capacity.

Where classical regularization (weight decay, dropout, spectral normalization) constrains the *mapping* $f : x \mapsto y$, TNN regularization constrains the *trajectory* $V(t)$. These mechanisms are complementary.

4 Experiments

We demonstrate TNN capabilities using human activity recognition as a representative temporal classification task. The stability and robustness improvements shown here are expected to transfer to other domains requiring robust temporal inference.

4.1 Dataset

We use the UCI Human Activity Recognition dataset with **proper subject-based splits**:

- Training: 7,352 samples from 21 subjects
- Testing: 2,947 samples from 9 subjects
- **Zero subject overlap**—no data leakage
- 6 activities: Walking, Walking Upstairs/Downstairs, Sitting, Standing, Laying
- 561 features extracted from accelerometer and gyroscope signals

4.2 Models

- **Classical**: [561, 128, 64, 6] feedforward network with tanh activation
- **Temporal**: Same architecture with leaky integration ($\tau = 8.0$, $dt = 1.0$)

Both models share identical weights—the temporal network is a direct conversion of the trained classical network.

4.3 Evaluation Protocol

For each experiment, we simulate streaming inference over 50 timesteps per sample. For noise experiments, independent Gaussian noise is added at each timestep. We measure:

- **Accuracy**: Fraction of correct final predictions
- **Prediction flips**: Average number of times the prediction changes across timesteps
- **Settle time**: Timesteps until prediction stabilizes

4.4 Results

4.4.1 Baseline Accuracy

Model	Accuracy	Macro F1
Classical	95.3%	0.952
Temporal	95.1%	0.951

Table 1: Baseline performance on clean data. TNN matches classical accuracy.

4.4.2 Stability Under Noise

We measure prediction flip rate—how often the model changes its prediction across consecutive timesteps under time-varying noise.

Noise σ	Classical Flips	TNN Flips	Reduction
0.0	0.0	0.9	—
0.2	1.1	0.9	16%
0.3	1.8	0.9	49%
0.5	3.7	0.9	75%
0.7	6.3	1.0	84%
1.0	11.0	1.0	91%

Table 2: Prediction flip rates under Gaussian noise. TNN shows 75–91% fewer flips at moderate to high noise levels.

Noise σ	Classical Acc	TNN Acc	Δ
0.0	98.3%	98.3%	0%
0.3	96.3%	99.0%	+2.7%
0.5	93.0%	99.0%	+ 6.0%
0.7	92.7%	99.0%	+6.3%
1.0	83.3%	97.7%	+ 14.4%

Table 3: Accuracy under noise. TNN is not just more stable—it is more accurate under noise.

4.4.3 Robustness to Missing Data

4.5 Ablation Studies

4.5.1 Time Constant Sweep

We evaluate the effect of the time constant τ on stability and accuracy trade-offs.

Key findings:

- $\tau = 1$ (near-instantaneous): High flip rate, lowest accuracy—insufficient smoothing
- $\tau = 2$ –4: Optimal range—low flips, high accuracy, fast settling
- $\tau = 8$ –16: Good stability but slower settling time

4.5.2 TNN vs Post-Hoc Smoothing

We compare TNN temporal dynamics against common post-hoc smoothing techniques applied to classical network outputs.

TNN achieves accuracy comparable to the best smoothing methods while providing several advantages:

- **No delay:** Post-hoc smoothing introduces lag; TNN dynamics are integrated into computation
- **Adaptive:** TNN settles faster when input is stable, slower under noise
- **State maintenance:** TNN preserves information across timesteps rather than just averaging outputs

Dropout %	Classical Acc	TNN Acc	Δ
0%	97.0%	96.8%	-0.2%
20%	94.2%	96.4%	+2.2%
30%	91.6%	95.4%	+3.8%
40%	86.0%	94.4%	+8.4%
50%	81.2%	89.0%	+7.8%
60%	76.0%	82.8%	+6.8%

Table 4: Accuracy with feature dropout. TNN degrades 33% more gracefully.

τ	Flips	Accuracy	Settle Steps
1	3.5	94.3%	13.2
2	0.3	97.7%	2.1
4	0.9	98.0%	2.3
8	1.2	98.0%	5.2
16	1.1	97.7%	11.9

Table 5: Effect of time constant τ on stability and accuracy under noise ($\sigma = 0.5$). $\tau = 4$ provides optimal balance of accuracy and responsiveness.

4.5.3 Comparison with Noise-Injection Training

We evaluate whether noise-injection during training provides similar benefits to TNN dynamics.

Key findings:

- Aggressive noise injection ($\sigma = 0.5$) matches TNN accuracy but with 82% more flips
- TNN provides stability benefits *without* requiring training modifications
- The approaches are complementary—noise-injection + TNN could be combined

5 Self-Tuning Latency Feedback

5.1 Motivation: the stability–responsiveness trade-off

The time-constant sweep (Table 5) exposes a tension inherent to any fixed- τ leaky integrator: a large τ gives stability and noise rejection but slow response, while a small τ gives fast response but admits jitter. The ablation’s “optimal” τ is a compromise chosen for a particular noise level. We now describe an *optional* mechanism that removes the need to choose: each neuron **self-tunes its own time constant online**, behaving like a large τ when the input is steady or noisy and like a small τ during a genuine, sustained change. It is opt-in and off by default, so all results above are unchanged.

5.2 Mechanism

The adaptation is a closed loop with two parts: a self-calibrating change *detector* and a tracking-lag *feedback* that gates the response.

Method	Flips	Accuracy
Classical (baseline)	3.4	96.0%
Moving Average ($k = 3$)	0.9	96.7%
Moving Average ($k = 5$)	0.7	97.3%
Moving Average ($k = 10$)	0.3	97.3%
Exp. Smoothing ($\alpha = 0.3$)	1.2	97.3%
Exp. Smoothing ($\alpha = 0.1$)	0.3	97.7%
TNN ($\tau = 8$)	1.1	97.7%

Table 6: TNN vs post-hoc smoothing under noise ($\sigma = 0.5$).

Training Method	Flips	Accuracy
Standard training	3.9	94.7%
Noise injection ($\sigma = 0.1$)	3.0	95.0%
Noise injection ($\sigma = 0.3$)	2.8	94.7%
Noise injection ($\sigma = 0.5$)	2.0	98.0%
TNN (standard training)	1.1	98.0%

Table 7: Noise injection training vs TNN under test noise ($\sigma = 0.5$).

Onset detector (self-calibrating). Let $u_t = g I_t$ be the raw input drive. We form a difference-of-EMAs band-pass and z-score it against an online estimate of its own variance:

$$m_t^{\text{fast}} = m_{t-1}^{\text{fast}} + r_f (u_t - m_{t-1}^{\text{fast}}), \quad m_t^{\text{slow}} = m_{t-1}^{\text{slow}} + r_s (u_t - m_{t-1}^{\text{slow}}), \quad (6)$$

$$b_t = m_t^{\text{fast}} - m_t^{\text{slow}}, \quad \nu_t = \nu_{t-1} + r_\sigma (b_t^2 - \nu_{t-1}), \quad (7)$$

$$z_t = |b_t| / (\sqrt{\nu_t} + \epsilon), \quad (8)$$

with per-step rates $r_f > r_s > r_\sigma$ (windows in steps). The band-pass b_t rejects steady DC (via the slow mean) and single-sample noise (via the fast mean); the online variance ν_t makes the significance z_t *scale-free*, expressed in units of the signal’s own noise. The only threshold, z_{thresh} , is therefore in sigma units and need not be retuned per signal—addressing the fact that there is no universal best *absolute* threshold.

Closed-loop response. A significant change latches a “chase”:

$$\tau_t^{\text{eff}} = \begin{cases} \tau_{\text{min}} & \text{while chasing} \\ \tau & \text{otherwise,} \end{cases} \quad (9)$$

where chasing is set when $z_t > z_{\text{thresh}}$ and cleared only when the change is fully accommodated—both the band-pass has subsided ($z_t < z_{\text{exit}}$) *and* the neuron’s output has caught up (output-space lag below a small tolerance). The output-space (saturating) lag is what *sustains* the chase until the neuron actually tracks the new level, and—because a one-step noise spike produces no real output lag—it also *terminates* spurious chases immediately. This feedback is what lets τ stay small exactly long enough, without a hand-tuned hold time.

5.3 Result: recovering both ends of the sweep

On a single neuron tracking a step embedded in heavy noise ($\sigma = 0.8$), the self-tuning neuron (default settings) keeps the steady-state jitter of the *large- τ* neuron (essentially zero) while cutting step-response latency substantially (e.g. ≈ 29 vs ≈ 69 steps in the reproducible bundled test), i.e. approaching the responsiveness of the *small- τ* neuron without its noise penalty. It thus recovers both ends of Table 5 from one self-calibrating unit, and the same configuration holds across signals of different scale and noise level because the trigger is in sigma units.

5.4 A second use: the self-tuning trace as a signal probe

Because the detector calibrates to the signal’s own noise floor, its internal trace is a free, label-free characterization of the input: the chase episodes mark *change-points*, the online $\sqrt{\nu_t}$ tracks the *noise floor*, and z_t separates real change from noise. In a multi-regime test the unit stays calm through a noisy-but-steady stretch (no false alarm) and fires only at genuine steps, while its noise-floor estimate rises in the noisy regime. This positions the mechanism between adaptive filtering and online change-point detection: it is a causal, $O(1)$ -state, self-calibrating adaptive low-pass whose change detector is *coupled* to the filter—the detector retunes the filter rather than merely reporting.

5.5 Limitation

A *very gradual* drift can evade the instantaneous significance test, since the drift’s own band-pass slowly inflates the online variance it is measured against. Robust slow-drift detection is a natural extension via CUSUM-style accumulation of the normalized band-pass; the present detector targets abrupt regime changes with strong noise rejection.

6 Discussion

6.1 Stability as a First-Class Property

The observed 75–91% reduction in prediction flips is not marginal—it represents a **qualitative behavioral difference**. The classical network oscillates under noise; the TNN maintains coherent predictions.

This directly addresses **alarm fatigue** in clinical settings, where flickering predictions cause:

- False alarms overwhelming clinicians
- Reduced trust in automated systems
- Potential patient safety issues

6.2 Why This Is Not “Just Smoothing”

Post-hoc temporal smoothing can reduce flips, but TNN provides additional benefits:

1. **Accuracy improvement:** TNNs improve accuracy under noise; pure smoothing does not
2. **State integration:** TNNs integrate information across time steps, not just average outputs
3. **No delay:** Smoothing introduces lag; TNN dynamics are simultaneous with computation
4. **Graceful degradation:** TNNs handle missing data through temporal integration

The ablation study (Table 6) shows that while heavy smoothing can match TNN’s flip reduction, TNN achieves comparable stability with better responsiveness and without post-processing overhead.

6.3 Relation to Classical Regularization

TNNs operate on a different axis than classical regularization:

- **Weight decay:** Constrains weight magnitudes
- **Dropout:** Regularizes co-adaptation of features
- **Spectral normalization:** Constrains Lipschitz constant
- **TNN dynamics:** Constrains trajectory volatility

These mechanisms are **complementary**. A network can use weight decay during training *and* TNN dynamics at inference. The noise-injection comparison (Table 7) shows that training-time regularization and inference-time dynamics provide independent benefits.

6.4 Biological Plausibility

The TNN equation (Eq. 1) is the **Leaky Integrate-and-Fire model** used throughout computational neuroscience [Gerstner and Kistler, 2002]. It captures key features of biological neurons:

- Integration of inputs over time
- Leak toward resting potential
- Temporal filtering of high-frequency noise

Real neural circuits trade instantaneous responsiveness for temporal stability—exactly what we observe in TNNs.

6.5 Limitations

- **Inference cost:** TNNs require multiple settle steps, increasing inference time
- **Hyperparameter:** The time constant τ must be chosen (though the ablation shows a broad optimal range)
- **Task scope:** Benefits are most pronounced for temporal/streaming tasks; less relevant for single-shot classification

7 Conclusion

Temporal Neural Networks demonstrate that modeling neurons as dynamical systems provides substantial practical benefits: matching classical accuracy while delivering dramatically improved stability (75–91% fewer prediction flips) and robustness (+8.4% accuracy at 40% dropout).

TNNs should be understood not as a new neural network family, but as a **temporalization transform**—a dynamical constraint layer applicable to existing feedforward architectures. By separating representation learning from temporal dynamics, TNNs offer a practical, interpretable, and computationally efficient approach to robust temporal inference.

The properties demonstrated—more stable decisions over time, fewer false alarms, and graceful degradation—are valuable across diverse applications:

- **Clinical monitoring:** ECG arrhythmia detection, EEG seizure prediction, patient vital signs
- **Industrial systems:** Predictive maintenance, anomaly detection, quality control
- **Autonomous systems:** Robotics, drone control, self-driving vehicles
- **Financial applications:** Fraud detection, algorithmic trading, risk assessment

Classical neural networks are *accurate but brittle*. Temporal neural networks are *accurate and well-behaved in time*.

Code Availability

Code is available at: <https://github.com/pcoz/temporal-neural-networks>

References

- Beer, R. D. On the dynamics of small continuous-time recurrent neural networks. *Adaptive Behavior*, 3(4):469–509, 1995.
- Bishop, C. M. Training with noise is equivalent to Tikhonov regularization. *Neural Computation*, 7(1):108–116, 1995.
- Chen, R. T., Rubanova, Y., Bettencourt, J., and Duvenaud, D. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, 2018.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *EMNLP*, 2014.
- De Brouwer, E., Simm, J., Arany, A., and Moreau, Y. GRU-ODE-Bayes: Continuous modeling of sporadically-observed time series. In *Advances in Neural Information Processing Systems*, 2019.
- Gerstner, W. and Kistler, W. M. *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press, 2002.
- Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- Hasani, R., Lechner, M., Amini, A., Liebenwein, L., Ray, A., Tschaikowski, M., Teschl, G., and Rus, D. Closed-form continuous-time neural networks. *Nature Machine Intelligence*, 4:992–1003, 2022.

- Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- Kidger, P., Morrill, J., Foster, J., and Lyons, T. Neural controlled differential equations for irregular time series. In *Advances in Neural Information Processing Systems*, 2020.
- Madry, A., Makelov, A., Schmidt, L., Tsipras, D., and Vladu, A. Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations*, 2018.
- Miyato, T., Kataoka, T., Koyama, M., and Yoshida, Y. Spectral normalization for generative adversarial networks. In *International Conference on Learning Representations*, 2018.
- Rifai, S., Vincent, P., Muller, X., Glorot, X., and Bengio, Y. Contractive auto-encoders: Explicit invariance during feature extraction. In *International Conference on Machine Learning*, 2011.
- Rubanova, Y., Chen, R. T., and Duvenaud, D. Latent ordinary differential equations for irregularly-sampled time series. In *Advances in Neural Information Processing Systems*, 2019.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1): 1929–1958, 2014.